

Table of Contents

- [1. Objective](#)
- [2. The file in one chunk](#)
- [3. Create the interface file and save it to Hello.h](#)
- [4. Create the implementation file Hello.cpp](#)
- [5. Create the test file](#)
- [6. Run the tangled files](#)
- [7. Create the Makefile](#)
- [8. Run the Makefile](#)
- [9. Run the executable](#)

1. Objective

Demonstrate how to write and test C++ source code spread out over multiple files with the help of make.

- [] The file in one chunk (hello.cpp)
- [] Create the interface file (Hello.h)
- [] Create the implementation file (Hello.cpp)
- [] Create the test file (HelloTest.cpp)
- [] Create the Makefile
- [] Run the Makefile with make
- [] Run the test file Hello

2. The file in one chunk

We have one C++ file that includes:

1. Interface with class declaration (Hello.h)
2. Implementation with class member function definitions and header files (Hello.cpp)
3. Test file with main function, Hello object creation and calls to the public member functions to tell us the name and say hello (HelloTest.cpp)

You can run this file with C-c C-c or you can tangle it with C-c C-v t (cursor in the code block), compile and run it on the command line.

```
#include <iostream> // I/O header file
#include <string>    // string class
using namespace std; // std namespace

// Interface
class Hello {
public:
    Hello(const string& n=""); // constructor
    string getName() const;    // accessor
    void sayHello() const;     // accessor
private:
    string name;
};

// Implementation
Hello::Hello(const string& n)
    : name(n) {}
string Hello::getName() const { return name; }
```

```

void Hello::sayHello() const { cout << "Hello, " << name << "!\n"; }

// Test function
void testHello()
{
    Hello hi("Dr. Birkenkrahe");
    cout << "You are: " << hi.getName() << endl;
    hi.sayHello();
}

// main program
int main()
{
    testHello();
    return 0;
}

```

```

cd ../src
ls -lt hello.cpp
g++ hello.cpp -o hello
./hello

```

3. Create the interface file and save it to Hello.h

None of these files will compile

```

#ifndef HELLO_H
#define HELLO_H

#include <string>
#include <iostream>

class Hello {
public:
    Hello(const std::string& n = "");
    std::string getName() const;
    void sayHello() const;
private:
    std::string name;
};

#endif

```

Tangle it with (org-babel-tangle)

```

cd ../src
ls -lt Hello*

```

```

-rw-rw-r-- 1 aletheia aletheia 238 Jul 18 22:00 HelloTest.cpp
-rw-rw-r-- 1 aletheia aletheia 240 Jul 18 21:59 Hello.cpp
-rw-rw-r-- 1 aletheia aletheia 226 Jul 18 21:58 Hello.h

```

4. Create the implementation file Hello.cpp

- Include "Hello.h" in this file.
- Use the header argument :flags "-I ../src" to find Hello.h

```
#include "Hello.h"
#include <iostream>
#include <string>
using namespace std;

Hello::Hello(const string& n) : name(n) {}

string Hello::getName() const { return name; }

void Hello::sayHello() const { cout << "Hello, " << name << "!\n"; }
```

Tangle it with (org-babel-tangle)

```
cd ../src
ls -lt Hello*
```

```
-rw-rw-r-- 1 aletheia aletheia 238 Jul 18 22:00 HelloTest.cpp
-rw-rw-r-- 1 aletheia aletheia 240 Jul 18 21:59 Hello.cpp
-rw-rw-r-- 1 aletheia aletheia 226 Jul 18 21:58 Hello.h
```

5. Create the test file

```
#include <iostream>
#include <string>
#include "Hello.h"
using namespace std;

void testHello() {
    Hello hi("Dr. Birkenkrahe");
    cout << "You are: " << hi.getName() << endl;
    hi.sayHello();
}

int main() {
    testHello();
    return 0;
}
```

6. Run the tangled files

```
cd ../src
ls -lt Hello*
```

```
-rw-rw-r-- 1 aletheia aletheia 238 Jul 18 22:00 HelloTest.cpp
-rw-rw-r-- 1 aletheia aletheia 240 Jul 18 21:59 Hello.cpp
-rw-rw-r-- 1 aletheia aletheia 226 Jul 18 21:58 Hello.h
```

```
cd ../src
g++ -c Hello.cpp -o Hello.o
ls -lt Hello*
```

```
-rw-rw-r-- 1 aletheia aletheia 3376 Jul 19 07:04 Hello.o
-rw-rw-r-- 1 aletheia aletheia  238 Jul 18 22:00 HelloTest.cpp
-rw-rw-r-- 1 aletheia aletheia  240 Jul 18 21:59 Hello.cpp
-rw-rw-r-- 1 aletheia aletheia  226 Jul 18 21:58 Hello.h
```

```
cd ../src
g++ HelloTest.cpp Hello.o -o Hello
ls -lt Hello*
```

```
-rwxrwxr-x 1 aletheia aletheia 24248 Jul 19 07:04 Hello
-rw-rw-r-- 1 aletheia aletheia  3376 Jul 19 07:04 Hello.o
-rw-rw-r-- 1 aletheia aletheia  238 Jul 18 22:00 HelloTest.cpp
-rw-rw-r-- 1 aletheia aletheia  240 Jul 18 21:59 Hello.cpp
-rw-rw-r-- 1 aletheia aletheia  226 Jul 18 21:58 Hello.h
```

```
cd ../src
./Hello
```

```
You are: Dr. Birkenkrahe
Hello, Dr. Birkenkrahe!
```

7. Create the Makefile

The action line after the target line needs to be indented: In Emacs, that works with C-q TAB.

```
# Makefile for Hello class and test
Hello: Hello.o HelloTest.o
    g++ -o Hello Hello.o HelloTest.o

Hello.o: Hello.cpp Hello.h
    g++ -c Hello.cpp -o Hello.o

HelloTest.o: HelloTest.cpp Hello.h
    g++ -c HelloTest.cpp

clean:
    rm -f *.o Hello
```

Here is a more generic version: You can test it with `make -f Makefile2` after tangling the Makefile.

```
# Makefile for CreditCard class and test
CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -g

TARGET   = Hello
OBSJS    = Hello.o HelloTest.o
```

```
$(TARGET): $(OBJS)
    $(CXX) $(CXXFLAGS) -o $(TARGET) $(OBJS)

Hello.o: Hello.cpp Hello.h
    $(CXX) $(CXXFLAGS) -c Hello.cpp

HelloTest.o: HelloTest.cpp Hello.h
    $(CXX) $(CXXFLAGS) -c HelloTest.cpp

clean:
    rm -f *.o $(TARGET)
```

```
cd ../src
ls -lt Make*
make -f Makefile2 clean
ls -lt Hello*
make -f Makefile2
./Hello
```

```
-rw-rw-r-- 1 aletheia aletheia 368 Jul 19 07:03 Makefile2
-rw-rw-r-- 1 aletheia aletheia 234 Jul 18 22:10 Makefile
rm -f *.o Hello
-rw-rw-r-- 1 aletheia aletheia 238 Jul 18 22:00 HelloTest.cpp
-rw-rw-r-- 1 aletheia aletheia 240 Jul 18 21:59 Hello.cpp
-rw-rw-r-- 1 aletheia aletheia 226 Jul 18 21:58 Hello.h
g++ -std=c++17 -Wall -Wextra -g -c Hello.cpp
g++ -std=c++17 -Wall -Wextra -g -c HelloTest.cpp
g++ -std=c++17 -Wall -Wextra -g -o Hello Hello.o HelloTest.o
You are: Dr. Birkenkrahe
Hello, Dr. Birkenkrahe!
```

8. Run the Makefile

```
cd ../src
ls -lt Makefile
make
```

```
-rw-rw-r-- 1 aletheia aletheia 234 Jul 18 22:10 Makefile
make: 'Hello' is up to date.
```

9. Run the executable

```
cd ../src
./Hello
make -f Makefile2 clean
```

```
You are: Dr. Birkenkrahe
Hello, Dr. Birkenkrahe!
rm -f *.o Hello
```